WINDOWS 桌面 AI 助理

Avalonia Assistant

一个能真正动手干活的本地桌面 AI 助理：流式对话、工具调用、技能与 MCP、三档权限、产物差异可见，会话全部留在本机。

项目属性	内容
产品形态	Windows 桌面应用（单窗口，1280×820，可缩放 90%–125%）
技术底座	.NET 10 · Avalonia 12.1.4 · TKCode.Arc（AtomUI 生态）· pi.net 1.4.4
工程结构	6 个 src 项目 + 1 个测试项目，171 个自动化测试
运行要求	Windows 10/11；框架依赖或自包含单文件（约 82 MB）发布
数据存放	%APPDATA%\AvaloniaAssistant（settings.json / sessions/*.json）
开源协议	MIT



首页：欢迎区、快捷提示、建议卡与统一输入框

01 项目定位

Avalonia Assistant 是一个跑在 Windows 桌面上的 AI 助理客户端。它把「聊天」和「干活」放在同一个窗口里：模型能读文件、改代码、跑命令、读写 Word / Excel，而每一步做了什么、改了哪些文件，用户都能在对话里当场看见。

三个设计取向

- **过程可见**：工具调用压成一行摘要（名称 + 耗时），点开才看参数与结果；文件改动单独列成产物清单，带逐行差异。
- **权限可控**：只读 / 工作区 / 完全访问三档，越界写入直接拦截，执行命令弹授权卡。
- **数据本地**：会话一个 JSON 一个文件，第一句话发出就落盘；不依赖任何云端账号体系。

与常见客户端的不同

维度	常见做法	本项目做法
工具执行	只展示最终回答，过程不可见	工具调用逐条入列，可展开参数与结果，失败原因直接显示
文件改动	让模型自己复述改了什么	Agent 侧在工具调用前后拍快照对比，产出真实行级 diff
权限	全局开关或完全信任	三档权限 + 逐次授权卡（允许一次 / 本会话允许 / 拒绝）
扩展	插件需要写代码	技能 = 一个 SKILL.md，MCP = 一段 JSON，界面内可导入
会话	存在云端，需要登录	本机 sessions/*.json，可导出 Markdown

02 核心能力

能力按「输入 → 执行 → 结果 → 记录」四条线组织。

输入：把上下文带进来

- **@ 引用文件**：输入框打 @ 按名字搜当前工作区，排序为文件名精确 > 前缀 > 包含 > 路径 > 字符子序列，自动跳过 node_modules / .git；选中即引用，发送时才读内容。
- **粘贴图片**：Ctrl+V 直接贴截图，缩略图挂在输入框上方随文字一起发；图片存数据目录 images/，会话 JSON 只记路径。
- **拖放**：文件拖到窗口任意位置都能接住，图片按图片加、其它文件走引用，文件夹与重复项各有提示。
- **语音输入**：NAudio 录音 + /audio/transcriptions 转写（端点需支持）。

执行：真的会动手

自建 8 个工具补足 CodingAgent 内置工具没有的能力，同一件事只留一个实现：

工具	作用	权限
write_file	写工作区文件，支持追加、自动建父目录	工作区可写 / 只读拦截
read_document	读 docx / xlsx / pptx / pdf，还原标题层级与表格为 Markdown	只读
write_document	把 Markdown 写成 .docx / .xlsx，标题、列表、表格、代码块落成真样式	工作区可写
update_plan	多步任务列计划并更新进度，对话里显示成清单	只读
open_path	用系统程序打开文件 / 文件夹（拒绝直接启动可执行文件）	只读
run_command	执行 shell 命令，60s 超时、进程树 kill、输出上限 20 万字符	需确认
fetch_url	抓网页正文	只读
current_time	当前时间与时区	只读

另有 CodingAgent 内置的 read / grep / find / ls / edit / write，以及 MCP 服务端暴露的 mcp__<server>__<tool> 工具，共 8 + 7 类可调用能力。

结果：看得见、拿得走

- **产物面板**：写过的文件在右侧列成清单，标出新建 / 修改 / 删除与增删行数，点开是逐行差异（+ 绿 − 红），可直接打开文件或在资源管理器定位。
- **计划清单**：模型调 update_plan 后，步骤以打勾 / 时钟 / 空心方块呈现，方向不对可以在第一步之前叫停。
- **办公文件产出**：read_document / write_document 让 docx、xlsx 成为一等公民，读进来是结构化 Markdown，写出去是带样式的真实文档。
- **导出会话**：对话页右上角一键存成 Markdown，工具调用、失败轮次、引用文件都在里面。

记录：会话与工作区

- **会话落盘**：一个会话一个 JSON，第一句话发出即写盘，不等回复跑完。
- **侧栏组织**：「任务」列全部任务并按今天 / 昨天 / 最近 7 天分组；「空间」只是按目录再归类一次，归了空间的任务在「任务」里照样可见。
- **工作区切换**：输入框下方胶囊点开是「搜索 + 工作区列表」，新建工作空间、打开本地文件夹、不使用工作空间都在同一个浮层里。普通任务的文件落在 ~/.avaloniaassistant。
- **状态记忆**：窗口位置尺寸、侧栏宽度、界面缩放、最后一条任务都会记住，关掉再开直接接上。
- **完成提醒**：窗口不在前台时一轮结束闪任务栏并响提示音，失败用警示音，可静音。



会话页：消息、工具卡、产物清单与统一输入框

03 技术架构

两个核心框架各管一边：**TKCode.Arc** 负责界面，**pi.net** 负责 Agent 循环，其余选型都是围着这两个补的。

层	选型	版本
UI 框架	TKCode.Arc（AtomUI 生态，控件 / 主题 / 图标字体）	0.3.1-preview.3
基础 UI	Avalonia	12.1.4
控件库	AtomUI.Desktop.Controls	6.2.8
Agent 框架	pi.net（Pi 的 .NET 移植，Pi.CodingAgent）	1.4.4
运行时	.NET	10.0
MVVM	CommunityToolkit.Mvvm	8.4.0
语音	NAudio	2.2.1

分层与解耦

依赖方向严格单向：**UI 与 Agent 互不引用**，只共同依赖零第三方依赖的 Abstractions 契约层；Desktop 是唯一同时认识两边的组合根。

项目	职责
AvaloniaAssistant.Abstractions	契约与 DTO：IAssistantRuntime、事件、会话、设置（零依赖）
AvaloniaAssistant.Agent	唯一认识 pi.net 的项目：会话缓存、事件映射、权限钩子、工具注册
AvaloniaAssistant.Storage	settings.json / sessions/*.json 原子写入，技能与 MCP 导入
AvaloniaAssistant.Speech	NAudio 录音 + 语音转写
AvaloniaAssistant.UI	窗口、页面、ViewModel、样式、截图模式（不引用任何 Pi.*）
AvaloniaAssistant.Desktop	入口、DI 组合根、Arc 初始化、命令行参数
AvaloniaAssistant.Agent.Tests	171 个测试：循环、工具、HTTP 流、持久化、技能导入、文档往返

换掉 Agent 框架只需另写一个 IAssistantRuntime 实现并在 Program.cs 替换注册；换掉 UI 框架，Agent / Storage 一行不用改。

一次对话的时序

用户输入 → ViewModel 组装 AgentRunRequest → Runtime 建会话并注册工具 → Pi 的 Agent 循环 → 事件流回 UI → 落盘。UI 与 Agent 之间唯一的通信面是 AgentStreamEvent。

事件	携带	UI 行为
RunStarted	sessionId / model	顶部状态改为「正在连接…」
TextDelta	增量文本	追加到 Markdown，55ms 节流批量刷新
ThinkingDelta	思考增量	折叠区「思考过程」
ToolStarted	callId / name / args	插入工具卡，状态「运行中」
ToolFinished	result / isError / duration	更新状态、耗时，可展开结果
FileChanged	一条文件改动记录	产物面板加一行，带行级 diff
RunCompleted	stopReason / token 用量	结束流式，写 token 脚注
RunFailed	message	红色错误块

文件改动为什么在 Agent 侧采集

事件走无界 Channel 异步消费，界面收到「工具开始」时文件往往已经写完，快照会拍到改之后的内容、diff 算出来是空的。因此采集器挂在 PiAgentRuntime 的工具执行同步回调里：前者拍改前快照，后者对比磁盘算差异，只认工具名白名单。

权限模型

模式	读工作区	写工作区	越界路径	执行命令
只读	允许	直接阻止	阻止	阻止
工作区	允许	允许	直接阻止	逐次授权
完全访问	允许	允许	允许	直接执行

实现分两层：描述层用 AgentToolInfo.IsMutating / RequiresApproval 标注；执行层在 BeforeToolCall 里按权限档位拦截，需要授权的调用走 IToolApprovalGate 弹卡片，用户拒绝则返回 Block。只读模式下连 edit 都不会出现在工具集里。

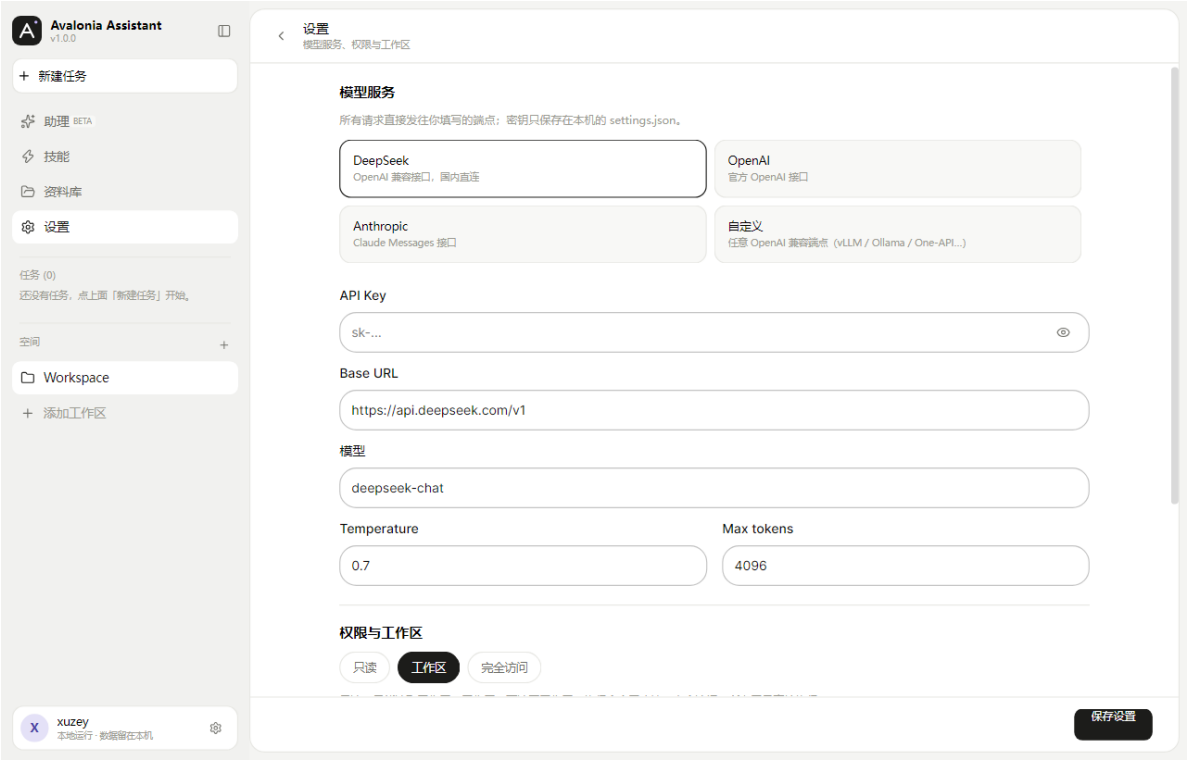
04 扩展方式

技能与 MCP 都按 pi 的目录约定发现，不需要写代码、不需要重新编译。

扩展点	位置	说明
项目级技能	<工作区>/.pi/skills/<名称>/SKILL.md	frontmatter 写 name / description，正文按需加载
全局技能	~/.pi/agent/skills/<名称>/SKILL.md	对所有工作区生效
项目级 MCP	<工作区>/.pi/mcp.json	启动时连接，需写 exposure: direct
全局 MCP	~/.pi/agent/mcp.json	同上
新增 provider	AgentProviderKind + ProviderPreset	UI 的模型卡片自动出现
新增工具	实现 IAgentTool 并注册	技能页与系统提示自动更新
新增页面	ViewModel + View + DataTemplate	在 MainWindowViewModel 挂命令

模型接入

支持 DeepSeek / OpenAI / Anthropic 以及任意 OpenAI 兼容端点（Ollama、LM Studio、vLLM、One-API）。多套渠道可以并存，各有独立的地址、Key 与协议，输入框下方胶囊随时切换；设置页的「测试连接」用界面上选中、可能还没保存的那套配置发一次最小请求并回显结果。



设置页：模型设置、权限档位、工作区与关于

05 质量保障与已知边界

自动化测试（171 个）

- Faux 模型驱动的完整 agent 循环与工具调用；只读拦截、授权拒绝、缺 API Key。
- 本机 TCP 模拟 OpenAI SSE 端点，验证真实 HTTP 流式解析与工具声明。
- MCP 端到端：测试里起一个 Node stdio 服务端，断言 mcp__demo__echo 进入可调用工具集。
- 文档往返：Markdown 写成 docx / xlsx 再读回来，标题层级、表格、项目符号、代码块逐项对齐。
- 技能导入：文件夹与 zip（外层目录、根级 SKILL.md、缺清单、多技能、重名）各走一遍。
- 文件工具的工作区沙箱、行级 diff 与计划文本解析、settings / session 的 JSON 往返。

UI 回归

界面不做自动化测试，改为提供 --screenshot 离线渲染：无需 API Key
即可输出首页、会话、设置、技能页，以及浮层、授权卡、产物面板、拖放提示层等 20
余个验收场景，用于文档配图与视觉回归。

已知边界（有意为之）

边界	说明
API Key 明文保存	settings.json 未接 DPAPI，共享机器上不要用生产密钥
只生成 docx / xlsx	pptx 需要母版与版式，宁可报错让人先出 docx 大纲
产物面板抓不到 run_command 改的文件	采集靠工具调用前后对比，shell 脚本改动无法定位
提示只有任务栏闪烁与提示音	未做托盘图标与系统通知，应用退出后无提醒
Windows 优先	代码对 macOS / Linux 做了分支，但只在 Windows 上验证
无定时任务 / 专家市场	导航里不留假按钮，扩展点见架构文档

文档索引

文档	内容
README.md	功能清单、快速开始、发布方式、内置工具、数据目录
docs/architecture.md	分层、时序、事件协议、权限、持久化、扩展点、线程模型
docs/research.md	pi.net 与 TKCode.Arc 的集成方式与踩坑记录
docs/migration.md	旧项目功能逐项盘点与取舍
docs/ux-review-*.md	三轮交互与视觉评审记录